

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY
Faculty of Technology and Engineering
Devang Patel Institute of Advance Technology and Research (DEPSTAR)
Department of Information Technology
ITUC301: Data Engineering
Practical Index

Practical No.	Practical Title
1	Understanding the Data Engineering Lifecycle
2	Data Sources and Data Generation
3	Batch vs Streaming Data Ingestion
4	ETL vs ELT Pipeline Design
5	Distributed Storage Systems
6	Storage Lifecycle Management
7	Change Data Capture (CDC)
8	Query Processing and Optimization
9	Data Modeling for Analytics
10	Batch and Streaming Transformations
11	Data Serving, Reverse ETL, and Security

Practical No		CO/PO
1	Problem Definition: Study, dissect, and map the complete end-to-end data engineering lifecycle for an enterprise e-commerce platform. Students must trace data states across all five core lifecycle stages: data generation, ingestion, storage, transformation, and serving. The solution requires creating a detailed architectural data flow diagram along with a system manifesto documenting cross-cutting lifecycle undercurrents (security, observability, and data privacy). This initial structure must be committed to a personal Git repository. Dataset/Test Data: Mock enterprise e-commerce dataset schema definition (containing transactional database entities, clickstream tracking JSON payloads, and third-party inventory log streams). Tools/Technology To Be Used: Draw.io, Lucidchart, Git, and GitHub/GitLab. Total Hours of Problem Definition Implementation: 2 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> How do the technical demands of downstream serving layers directly influence ingestion and storage design decisions during the early generation phases? • <i>Que / Key Point 2:</i> Differentiate between the broader data lifecycle and the technical data engineering lifecycle within an enterprise. • <i>Que / Key Point 3:</i> Identify how the undercurrents of security and data observability should be enforced at each lifecycle boundary. Supplementary Problems: Map out the data engineering lifecycle for a smart city IoT traffic sensor platform, emphasizing challenges related to continuous real-time streaming data generation. Key Skills to be addressed: System-level architectural thinking, functional data pipeline mapping, operational trade-off evaluation, and professional version control management. Applications: Enterprise business intelligence infrastructure, data platform modernizations, and cloud data platform layout planning. Learning Outcome: Ability to clearly identify, trace, and document structural data transitions across the complete data engineering lifecycle while balancing operational undercurrent constraints. Post Laboratory Work Description: Draft a comprehensive markdown README.md file detailing your system lifecycle selections, committing it directly to your newly initialized laboratory repository root directory.	CO1 / PO1, PO2, PO12, PSO1

2	Problem Definition: Programmatically interface with diverse file-based, API-driven, and relational source database environments. Students will write generation engines to simulate transactional workloads, profile the incoming files to automatically discover schemas, isolate data quality violations (such as null entry leaks and format divergence), and establish appropriate operational ingestion criteria. Dataset/Test Data: Programmatically simulated customer profile tables, semi-structured API transaction logs containing nested objects, and unstructured configuration text files. Tools/Technology To Be Used: Python 3.x, Faker library, Pandas, Requests HTTP library, SQLite/PostgreSQL, and Git. Total Hours of Problem Definition Implementation: 2 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs. • <i>Que / Key Point 2:</i> What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records? • <i>Que / Key Point 3:</i> How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks? Supplementary Problems: Develop a validation routine that parses a directory of messy user files, auto-quarantines records missing structural primary keys, and sends automated warning alerts to system logs. Key Skills to be addressed: Programmatic data profiling, synthetic record generation, automated quality boundary enforcement, and source system type evaluation. Applications: Source data profiling layers, raw data landing zones, and automated incoming payload quality gates. Learning Outcome: Proficiency in analyzing, classifying, and clean-profiling diverse transactional source data models prior to target system ingestion. Post Laboratory Work Description: Export your automated data quality execution logs alongside a detailed markdown summary outlining your strategies for validating malformed input fields.	CO1, CO4 / PO1, PO4, PO5, PSO2
3	Problem Definition: Design, spin up, and evaluate two parallel data ingestion pipelines: a periodic time-triggered block batch pipeline and a continuous low-latency real-time event streaming processor. Students must capture telemetry statistics and benchmark performance parameters across both options, measuring data delivery latency, processing throughput spikes, and fault tolerance during simulated infrastructure drops. Dataset/Test Data: Real-time web application user activity logs, server performance	CO3 / PO3, PO4, PO5, PSO1

	metrics, and timestamps. Tools/Technology To Be Used: Docker, Apache Kafka / Redpanda distributed brokers, Python (Kafka-Python-NG API client wrapper), and Pandas. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> In what scenarios does a micro-batch architecture provide a better operational balance of performance and complexity than a true real-time streaming approach? • <i>Que / Key Point 2:</i> How does an event streaming broker maintain message delivery guarantees during abrupt broker cluster node disconnects? • <i>Que / Key Point 3:</i> Detail the network and compute resource trade-offs between bulk file compression operations and individual event serialization overheads. Supplementary Problems: Extend your streaming pipeline to handle sudden traffic spikes by scaling your message broker topic partition layers and verifying how consumer groups scale out to process the load. Key Skills to be addressed: Streaming cluster container orchestration, real-time message consumer logic, bottleneck benchmarking, and fault-tolerance architecture evaluation. Applications: High-frequency log collection architectures, real-time security incident monitoring dashboards, and transactional synchronization feeds. Learning Outcome: Clear architectural and practical understanding of when to deploy batch versus streaming ingestion strategies based on latency, throughput, and system limits. Post Laboratory Work Description: Build a clear comparative performance chart evaluating processing latency profiles under variable workloads for both ingest frameworks, and commit your scripts.	
--	---	--

4	Problem Definition: Implement two different integration pipelines handling identical transaction data flows: an ETL (Extract, Transform, Load) pipeline that normalizes data in application memory before saving, and an ELT (Extract, Load, Transform) pipeline that lands raw data directly into a database staging area and leverages the target warehouse database engine to process transformations. Measure performance differences, data lineage, and maintenance requirements for both architectures. Dataset/Test Data: High-volume un-normalized retail sales record tables filled with inconsistent white spaces, raw numeric inputs, and legacy store codes. Tools/Technology To Be Used: Python, Pandas dataframe vector blocks, DuckDB analytical engine databases, and local file storage layers. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> How does the separation of compute and storage in modern cloud data lakes shift the architectural preference from classic ETL models toward ELT pipelines? • <i>Que / Key Point 2:</i> Compare your pipelines in terms of maintaining robust data lineage tracking and recovering from intermediate transformation bugs. • <i>Que / Key Point 3:</i> Evaluate how network resource costs shift when running transformation calculations inside application compute nodes versus inside database storage engines. Supplementary Problems: Refactor your ELT pipeline sql script into modular, testable views using dbt (Data Build Tool) core features, and implement automated schema testing assertions. Key Skills to be addressed: In-memory dataset transformations, database engine computation optimization, schema refactoring design patterns, and architectural cost analysis. Applications: Central data warehouse builds, scalable cloud data platform migrations, and decoupled analytic transformations. Learning Outcome: Ability to evaluate, implement, and choose between ETL and ELT paradigms based on infrastructure capabilities, system scale, and team requirements. Post Laboratory Work Description: Write an architectural design document summarizing your findings regarding execution speed and processing efficiency differences between your ETL and ELT runs.	CO2, CO3 / PO2, PO3, PO4, PO5
5	Problem Definition: Implement data storage structures using distributed storage	CO2 / PO2,

	paradigms. Students will partition large datasets using structured prefix layouts (Hive-style keys) within an S3-compatible object storage server, write programs to query specific partitions, and evaluate performance against full-table linear file scans. Dataset/Test Data: Multi-year analytical application metric tracking tables, partitioned by chronological timestamps and geographical regional identifiers. Tools/Technology To Be Used: MinIO Object Store Server (deployed via Docker multi-node containers), AWS Boto3 SDK core client tools, and Python. Total Hours of Problem Definition Implementation: 2 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Explain how distributed object storage architectures achieve high horizontal availability and durability compared to classical block-based storage frameworks. • <i>Que / Key Point 2:</i> Why do storage key paths (like <code>year=/month=</code>) serve as a primary query performance driver in analytical architectures that separate compute from storage? • <i>Que / Key Point 3:</i> How do distributed object stores handle high write concurrency without risking data visibility corruption or conflicts? Supplementary Problems: Build a file metadata crawling utility that scans your object storage buckets and calculates exact data balance or skew metrics across all active storage partitions. Key Skills to be addressed: Distributed storage setup, partition design layout optimization, object key namespace patterns, and remote programmatic storage consumption. Applications: Enterprise cloud data lake environments, cold archive storage clusters, and high-throughput data processing systems. Learning Outcome: Practical mastery over configuring and interacting with modern distributed storage engines while organizing data to maximize query isolation. Post Laboratory Work Description: Save your container compose setups, bucket partition mapping rules, and verification search scripts into a reusable storage module within your lab repository.	PO3, PO5, PSO1
6	Problem Definition: Formulate and test an automated storage lifecycle policy matrix defining three distinct access tiers: Hot (immediate access), Warm (infrequent access), and Cold (deep compression archiving). Students will implement programmatic policy rules that automatically transition data objects across these tiers based on retention targets, query frequency boundaries, and age constraints to minimize operational platform costs. Dataset/Test Data: Historical system operational logs, access time manifests, and server	CO2, CO5 / PO2, PO3, PO12

	audit records stretching back across multiple simulated quarters. Tools/Technology To Be Used: MinIO Client CLI engine binaries, JSON Lifecycle Policy schemas, and Python background automation scripts. Total Hours of Problem Definition Implementation: 2 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Detail the hardware and software properties that differentiate high-throughput Hot tier storage pools from highly compressed Cold tier archiving solutions. • <i>Que / Key Point 2:</i> What are the typical operational and financial cost penalties incurred when an enterprise misclassifies historical active data into Cold states prematurely? • <i>Que / Key Point 3:</i> Explain how automated expiration policies safeguard database systems against structural performance degradation over long operational windows. Supplementary Problems: Create an automation script that evaluates data access logs, flags data tables that have not been read for more than 90 days, and automatically demotes their storage allocation tier via API calls. Key Skills to be addressed: Automation policy authoring, lifecycle management rule testing, tiered cost optimization planning, and multi-tier data path validation. Applications: Automated compliance data archiving engines, tiered cost-conscious data lake platforms, and historical log rotation systems. Learning Outcome: Ability to design and enforce automated tiered data lifecycle policies that optimize storage costs while keeping active data highly performant. Post Laboratory Work Description: Commit your tested JSON lifecycle policy schemas and a comprehensive breakdown of estimated cost reductions achieved through tiering.	
7	Problem Definition: Simulate an incremental Change Data Capture (CDC) loop to safely track and capture all modifications (INSERT, UPDATE, DELETE operations) occurring on an operational transaction system. These captured operational row mutations must be streamed down into an audit logging ledger and correctly replayed onto an analytical table to maintain data consistency without running destructive full-table database dumps. Dataset/Test Data: Live application inventory database transactions subject to rapid stock updates, new order entry inserts, and row purge requests. Tools/Technology To Be Used: PostgreSQL container instances, Python (Psycopg2 driver integration), PL/pgSQL event logs, and message replication loops. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation:	CO3 / PO3, PO4, PO5, PSO1

	• <i>Que / Key Point 1:</i> Contrast the background resource footprint of log-based change data capture mechanisms against classic query-based scheduling checks on production tables. • <i>Que / Key Point 2:</i> How does your downstream replication engine handle network connection failures without losing event order or creating duplicate data writes? • <i>Que / Key Point 3:</i> What role does the transaction write-ahead log (WAL) fill in enabling real-time zero-impact CDC pipelines in production? Supplementary Problems: Implement a downstream deduplication logic check that handles out-of-order CDC updates by tracking database row version numbers. Key Skills to be addressed: Transaction database log auditing, system mutation event routing, transaction ordering verification, and downstream state replication. Applications: Zero-downtime database migrations, production operational-to-analytical synchronization layers, and auditable transaction stream monitoring. Learning Outcome: Practical understanding of building low-overhead, incremental change synchronization pipelines that connect transactional systems to analytical layers. Post Laboratory Work Description: Document the internal trigger or log-parsing mechanisms used in your implementation, and push your transaction capture modules to your lab repository.	
8	Problem Definition: Execute complex analytical aggregation queries across high-cardinality multi-million row datasets and inspect their internal database execution engines. Students must diagnose execution bottlenecks (such as full table linear disk scans or sorting overheads) using execution plan tools, and apply performance fixes like indexing, data partitioning, and query refactoring. Dataset/Test Data: Massive denormalized clickstream tracking log dataset containing millions of unstructured user engagement elements. Tools/Technology To Be Used: DuckDB Query Plan Analyzers or PostgreSQL engine databases, and Python analytical tools. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Explain what a full table disk scan operation means within an execution plan, and describe its computational impact on large datasets. • <i>Que / Key Point 2:</i> How does a columnar storage layout fundamentally change query processing and optimization compared to a traditional row-oriented layout? • <i>Que / Key Point 3:</i> What are the architectural costs and performance penalties associated with creating too many secondary indices on volatile, frequently updated	CO4 / PO4, PO5, PSO2

	database tables? Supplementary Problems: Take a highly nested SQL query containing multiple subqueries, profile its execution plan, and rewrite it using optimized Common Table Expressions (CTEs) or window functions to reduce execution costs. Key Skills to be addressed: Execution plan profiling, analytical index design, query optimization, and memory bottleneck identification. Applications: Database query tuning, warehouse cost optimization, and real-time dashboard data acceleration. Learning Outcome: Ability to profile, diagnose, and optimize database execution plans to build highly performant, scalable analytical systems. Post Laboratory Work Description: Include a before-and-after comparison report showcasing execution plans, query runtimes, and index performance metrics within your lab repository.	
9	Problem Definition: Design and implement a robust logical and physical dimensional data model tailored for business analytics. Students will transform transactional operational records into a classic Kimball Star Schema model, explicitly breaking fields out into highly descriptive Dimension tables and organizing numeric metrics within a central Fact table. All modeling choices must be justified based on expected query styles and joining efficiencies. Dataset/Test Data: Unstructured transactional sales lists containing repeated values, customer details, product categories, and order parameters. Tools/Technology To Be Used: dbdiagram.io, structural SQL Data Definition Language (DDL) syntax compilers, and DuckDB analytical engines. Total Hours of Problem Definition Implementation: 2 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> What are the key architectural differences and use cases for normalized operational models (3NF) versus denormalized analytical star schemas? • <i>Que / Key Point 2:</i> Explain the concept of Slowly Changing Dimensions (SCDs) and specify how your design tracks customer state changes over time. • <i>Que / Key Point 3:</i> How do surrogate keys protect an analytical data warehouse model from changes or migrations in source system business keys? Supplementary Problems: Expand your star schema design into a Snowflake layout by normalizing complex hierarchies within your Dimension tables, and compare the two structures in terms of storage size and join query performance. Key Skills to be addressed: Analytical dimension modeling, relationship definition,	CO4 / PO3, PO5, PSO2

	constraint optimization, and logical-to-physical model translation. Applications: Enterprise data mart architecture, analytical data warehouse design, and semantic layer visualization planning. Learning Outcome: Ability to construct scalable, analytics-ready dimensional models that simplify query complexity and maximize analytical calculation performance. Post Laboratory Work Description: Save your visual entity-relationship diagrams (ERD) along with the complete compiled SQL DDL scripts to set up the database structure.	
10	Problem Definition: Implement comprehensive data transformation modules across both batch environments and streaming runtime pipelines. Transformation logic must handle advanced operations including conditional data filtering, cross-source business enrichment joins, multi-level rolling window aggregations, and managing late-arriving stream records via system watermark constraints. Dataset/Test Data: Chronologically out-of-order IoT device telemetry records, some arriving significantly later than their actual event timestamps. Tools/Technology To Be Used: Apache Spark (PySpark platform libraries), Pandas, and Python datetime optimization packages. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Explain how a streaming engine uses watermarks to differentiate between acceptable late-arriving data and obsolete records that should be dropped. • <i>Que / Key Point 2:</i> Compare the processing mechanics and memory footprints of a rolling window aggregation against a standard tumbling window block in streaming pipelines. • <i>Que / Key Point 3:</i> What are the primary structural and operational challenges encountered when joining a fast real-time event stream with a slow batch metadata table? Supplementary Problems: Build a stream transformation block that detects fraud by calculating rolling, windowed transaction volume checks per user ID over 5-minute intervals. Key Skills to be addressed: PySpark framework usage, windowed aggregation design, watermark tracking setup, and multi-source dataset enrichment. Applications: Real-time financial fraud alerting engines, operational IoT telemetry cleaning pipelines, and streaming business metrics generation. Learning Outcome: Practical mastery over writing transformation logic for both batch and streaming pipelines while handling data arrival delay anomalies.	CO4 / PO3, PO4, PO5, PSO2

	Post Laboratory Work Description: Save your transformation code modules to your repository, including test logs showing how late-arriving records are dropped once they pass the watermark.	
11	Problem Definition: Create a secure analytical serving platform that shares data with downstream users, and build a Reverse ETL workflow to sync computed user metrics back to operational operational tools. The architecture must implement robust security controls, including column-level data encryption, SHA-256 identity hashing, database Role-Based Access Controls (RBAC), and centralized operation log tracking. Dataset/Test Data: Customer analytics output tables containing sensitive email fields, purchase segments, and churn prediction scores. Tools/Technology To Be Used: SQLite/PostgreSQL security controls, Airbyte/Meltano orchestration concepts, Python hashlib frameworks, and logging engines. Total Hours of Problem Definition Implementation: 4 Hours Key Questions / Analysis / Interpretation to be evaluated during/after Implementation: • <i>Que / Key Point 1:</i> Why do organizations deploy Reverse ETL workflows to sync analytical insights back to production tools instead of having production apps query warehouses directly? • <i>Que / Key Point 2:</i> How do column-level hashing and tokenization techniques help an enterprise meet strict privacy data regulations like GDPR or CCPA? • <i>Que / Key Point 3:</i> Detail the architectural differences between exposing data to analytical clients via direct views, data virtualization layers, and pre-computed data marts. Supplementary Problems: Implement a database access control layer that restricts data views so that members of a simulated data analyst role can only see masked or tokenized versions of sensitive customer identity columns. Key Skills to be addressed: Reverse ETL data activation, cryptography implementation, access control configuration, and comprehensive operational audit logging. Applications: Automated customer segment synchronization, secure analytical data serving platforms, and regulatory compliance data masking. Learning Outcome: Ability to design secure data serving architectures and implement operational synchronization pipelines that protect data privacy and adhere to compliance regulations. Post Laboratory Work Description: Commit your secure reverse ETL synchronization pipeline scripts alongside sample execution logs that capture system audit records and tokenized output fields.	CO5 / PO2, PO3, PO11, PO12, PSO1